

ГУАП

КАФЕДРА № 44

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

канд. техн. наук,
должность, уч. степень, звание

подпись, дата

Н.А. Соловьева
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Разработка диаграммы классов

по курсу: Технология программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4341В

подпись, дата

Р.С. Кулаков
инициалы, фамилия

Санкт-Петербург 2025

1 Вариант задания

В качестве разрабатываемой системы выбрана предметная область с прошлого семестра – Облачное хранилище файлов

2 Словесное описание классов, объяснение их роли в системе (не менее 0.5 листа, шрифт 12, одинарный интервал между строк)

2.1 FileMetadata

Хранит служебную информацию о файле (тип контента, теги). Используется при загрузке/проверке файла для оценки размера и классификации; метод `getSize()` возвращает размер метаданных/контента для проверки квоты и валидации.

2.2 ScanResult

Результат работы антивируса: булево поле `infected` и список обнаруженных угроз. Метод `isClean()` упрощает проверку результата — используется контроллером и сервисами для принятия решения о сохранении или удалении файла.

2.3 IVirusScanner (интерфейс)

Контракт для антивирусных реализаций — определяет метод `scan(file: File): ScanResult`. Позволяет подменять конкретный сканер

2.4 User

Модель пользователя системы: идентификатор, e-mail, имя и хеш пароля. Представляет владельца файлов и инициатора операций (загрузка, скачивание); на уровне модели здесь не прописаны методы, но экземпляры `User` используются в связях (`owner` файлов).

2.5 Admin

Специализированный пользователь — наследник `User`. Содержит административные операции, например `manageQuota(...)` для изменения ограничений хранения у пользователей. Наследование показывает, что админ обладает всеми свойствами обычного пользователя плюс расширенные права.

2.6 File

Центральный объект предметной области: метаданные о файле (`id`, имя, размер), ссылка на владельца и список версий (`versions`). `File` агрегирует версии как части своей истории; служит основой для операций хранения, шаринга и проверки безопасности.

2.7 FileVersion

Версия файла — внутри неё хранится полный двоичный контент (`data: byte[]`), контрольная сумма и номер версии. Наличие версий реализует хранение истории изменений и откат к предыдущим состояниям; проверка `checksum` гарантирует целостность.

2.8 Folder

Коллекция файлов: содержит идентификатор, имя и список ссылок на `File`. Реализована как агрегация — папка организует файлы, но файлы могут существовать независимо (перенос/удаление папки не обязательно уничтожает файлы).

2.9 StorageQuota

Описывает квоту пользователя (используемое и лимитное пространство) и операции управления — `check`, `reserve`, `release`. Используется сервисом хранения для принятия решения о возможности загрузки и для учета занимаемого места.

2.10 SharingLink

Модель публичной ссылки на файл: URL, срок действия и связанный File. Метод `isValid()` проверяет истечение срока, `revoke()` — аннулирует ссылку; используется для реализации шаринга без прямого раскрытия учётных данных владельца.

2.11 FileController

Оркестратор бизнес-логики: у него есть зависимости на `StorageService` и `IVirusScanner`. Методы `uploadFile`, `downloadFile`, `deleteFile` реализуют высокоуровневые сценарии — проверку квоты, сканирование и вызовы сервисов хранения; контроллер координирует обмен между хранилищем и внешними компонентами.

2.12 StorageService

Сервис, управляющий хранением: хранит сведения о квотах (`quotaStore`) и обеспечивает операции сохранения версии, получения и удаления файла. Именно он выполняет низкоуровневые операции (резервирование места, физическое сохранение версии) и отвечает за согласованность данных.

2.13 VirusScanner

Конкретная реализация `IVirusScanner`. Содержит версию движка и реализует `scan`, возвращающий `ScanResult`. Может быть как локальным модулем, так и прокси на внешний антивирусный сервис.

3 Диаграмма классов

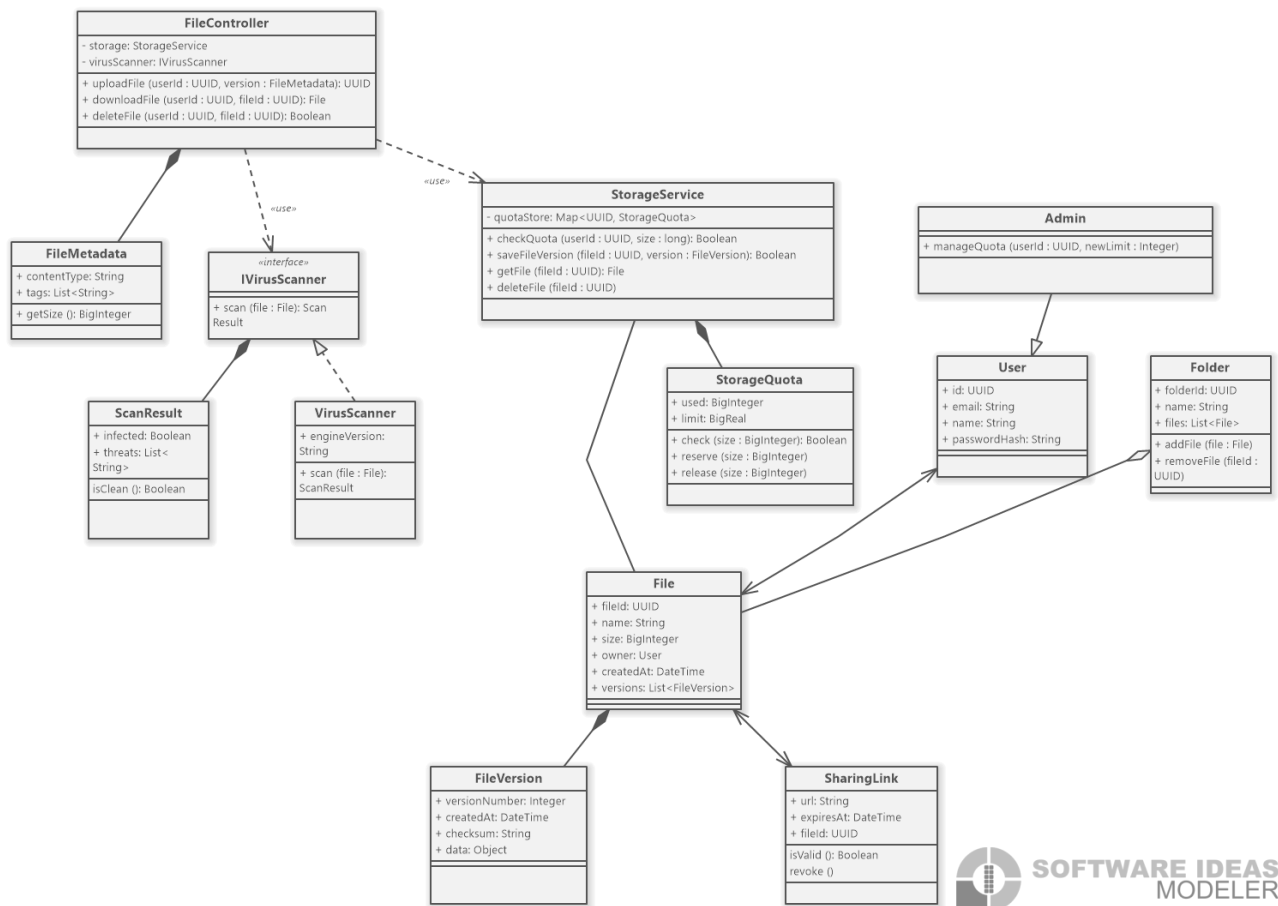


Рисунок 1 - Диаграмма классов

4 Анализ выполненной работы (комментарии по использованным отношениям между классами, атрибутам и методам классов)

4.1 Используемые отношения и атрибуты

4.1.1 Обобщение (Admin —|> User)

Админ — это частный случай пользователя, он наследует идентификационные поля. Показывает, что администратор обладает теми же данными, но дополнительными привилегиями (manageQuota).

4.1.2 Композиция (File -- FileVersion)

Правильно моделирует тот факт, что версии файла логически принадлежат файлу: версии не существуют отдельно. В File есть поле versions, что подтверждает композицию.

4.1.3 Агрегация (Folder o-- File)

Папка организует ссылки на файлы, но файлы могут существовать вне папки. Это отражает идею «коллекции» без владения.

Рекомендация: указать мультипликативность (например, Folder 1 — 0..* File)

4.1.4 Ассоциация (User -- File, File -- SharingLink, StorageService -- File)

Наличие поля owner в File подтверждает ассоциацию. SharingLink связано с File логично. StorageService ассоциирован с File как хранилище — это отражает реалистичную зависимость.

4.1.5 Зависимость (FileController ..> StorageService, ..> IVirusScanner, ..> Journal)

Контроллер использует сервисы. Стоит уточнить в методах контроллера параметры, где явно присутствуют объекты зависимых классов (например, uploadFile(userId, version) вызывает storage.saveFileVersion(...), scanner.scan(...)). Это подтвердит семантику dependency.

4.1.6 Реализация интерфейса (IVirusScanner <|.. VirusScanner)

Интерфейс задаёт контракт, реализация его выполняет.